

CHAPTER · ENSEMBLE II

Boosting: AdaBoost 에서 Gradient Boosting 까지

약한 모델을 “순차적으로” 쌓아 올려 , 앞 모델이 틀린 부분을 다음 모델이 보완한다

학습 목표

1 순차적 학습

Bagging/RF(병렬 , 분산 감소) 와 Boosting(순차 , bias 감소) 의 차이를 설명할 수 있다 .

2 AdaBoost.M1

잘못 분류된 관측치의 가중치를 키우는 과정과 , 모형별 가중 투표 (α) 를 이해한다 .

3 경사하강법으로 보는 Boosting

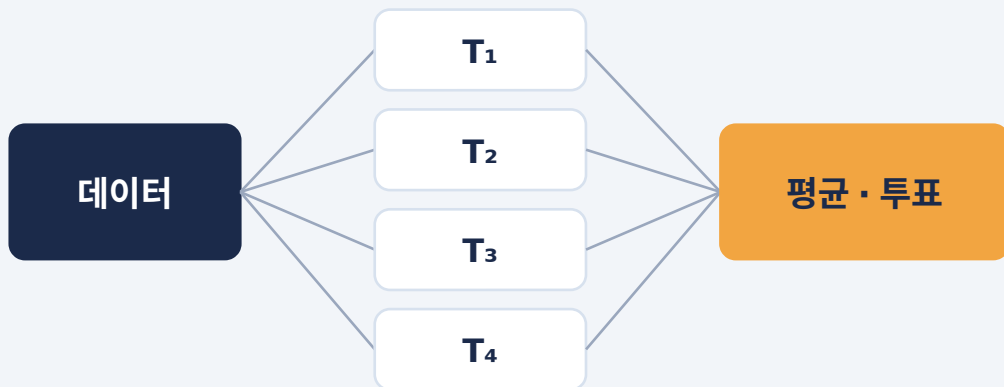
“ 어디서 시작해 손실이 작아지는 쪽으로 이동 ” 하는 local search 로 Gradient Boosting 을 이해한다 .

4 잔차의 반복 적합

회귀와 분류에서 손실함수의 기울기가 “잔차” 가 됨을 보이고 , gbm 으로 실습한다 .

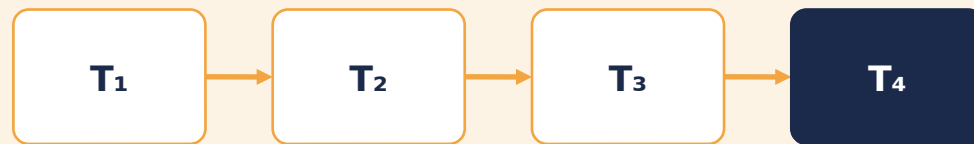
Bagging/RF 는 병렬 , Boosting 은 순차

Bagging · Random Forest



- 트리들을 서로 독립적으로 (동시에) 만든다
- 깊은 트리 → 평균으로 분산 (variance) 감소

Boosting



앞 모형이 틀린 부분 → 다음 모형이 보완

- 트리를 하나씩 순서대로 (sequential) 만든다
- 약한 (단순한) 트리를 쌓아 → 편향 (bias) 감소

비유 : Bagging = 여러 명이 각자 풀고 답을 모으기 , Boosting = 한 명이 오답노트를 만들어 틀린 문제를 집중적으로 다시 공부하기

순차적 학습이란 ? 약한 모델을 조금씩 쌓는다

1 순차적 (sequential)

m 번째 모형은 1, ..., m-1 번째 모형의 결과 (무엇을 틀렸는지) 를 보고 만들어진다 . 모형들이 서로 독립이 아니다 .

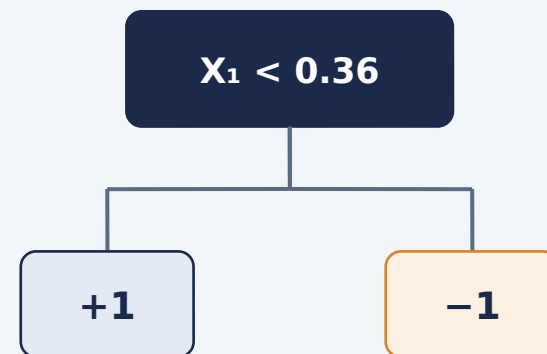
2 약한 학습기 (weak learner)

무작위 추측보다 “조금” 나은 단순한 모형 . 예 : 분할이 한 번뿐인 트리 (stump), 깊이 2~4 의 작은 트리

3 더하기로 만드는 강한 모형

$f(x) = \alpha_1 G_1(x) + \alpha_2 G_2(x) + \dots + \alpha_M G_M(x)$. 약한 모형 여러 개의 가중합 $\rightarrow$ 복잡한 패턴도 표현 가능

Stump (그루터기) = 깊이 1 트리



혼자서는 매우 부정확 (높은 bias). 하지만 수백 개를 순차적으로 쌓으면 강력해진다

Boosting 은 “약한 모형들을 합쳐 강한 모형을 만들 수 있는가 ?” 라는 질문에서 출발 (Schapire, 1990 · Freund & Schapire, 1997)

AdaBoost.M1 의 핵심 아이디어 : 관측치 가중치

Adaptive Boosting — 분류 문제, $Y \in \{-1, +1\}$

1

모든 관측치 동일한
가중치로 시작

$$w_i = 1/n$$

2

첫 번째 약한 모형 적합

일부 관측치를 틀린다

3

틀린 관측치의 가중치 $\uparrow$,
맞힌 관측치 $\downarrow$

“다음엔 이걸 꼭 맞혀라”

4

다음 모형은 가중치를
반영해 적합

무거운 관측치를 맞히려
노력

5

반복 후 가중 투표로 결합

잘한 모형일수록 큰 표 (α)

오답노트 비유

틀린 문제에 별표 (가중치) 를 하나씩 더 붙여 가며 다시 공부한다 . 자꾸 틀리는 어려운 문제일수록 별표가 쌓여 다음 공부에서 더 집중하게 된다 .

AdaBoost.M1 알고리즘

초기화

$$w_i = 1/n, i = 1, \dots, n$$

$m = 1, \dots, M$ 반복

(a) 가중치 w_i 를 사용해 분류기 $G_m(x)$ 적합

(b) 가중 오분류율 $err_m = \sum w_i I(y_i \neq G_m(x_i)) / \sum w_i$

(c) 모형 가중치 $\alpha_m = \log[(1 - err_m) / err_m]$

(d) 가중치 갱신 새 $w_i = w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))]$

최종 분류기

$$G(x) = \text{sign}[\sum_m \alpha_m G_m(x)]$$

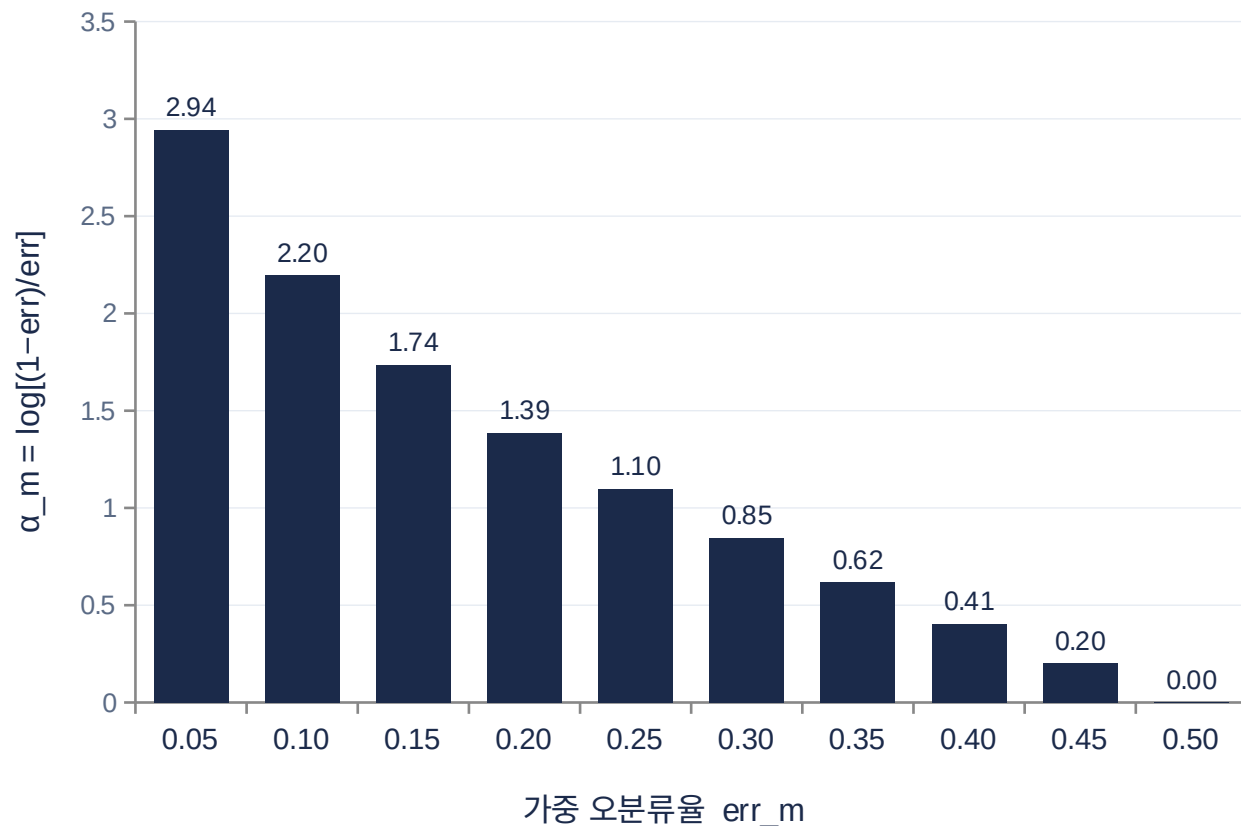
읽는 법

(c) 오류가 작은 모형 $\rightarrow \alpha$ 큼
(큰 발언권)

(d) 틀린 관측치만 e^α 배.
맞힌 관측치는 그대로 $\rightarrow$
정규화하면 상대적으로 감소

$I(\cdot)$: 조건이 참이면 1, 아니면 0

모형 가중치 α : 잘하는 모형이 더 큰 표를 갖는다



err = 0.05

$\alpha \approx 2.94$ · 매우 큰 발언권

err = 0.30

$\alpha \approx 0.85$

err = 0.50

$\alpha = 0$ · 동전 던지기 수준 → 표 없음

약한 학습기의 조건 : **err < 0.5** (무작위보다 나은 것)

같은 α 가 (d) 에서 틀린 관측치 가중치의 배율 e^α 로도 쓰인다: $err = 0.2 \rightarrow \alpha = 1.39 \rightarrow$ 틀린 관측치 가중치 $\times 4$

Bagging 은 1 인 1 표 , Boosting 은 가중 투표

Bagging / Random Forest

T ₁		1 표
T ₂		1 표
T ₃		1 표
T ₄		1 표
T ₅		1 표

모든 트리가 똑같은 발언권

AdaBoost

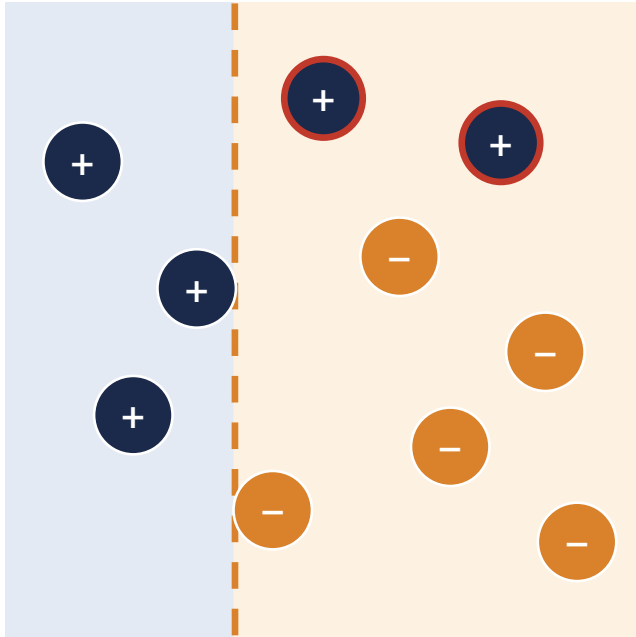
G ₁		$\alpha = 1.39$ (err 0.2)
G ₂		$\alpha = 1.95$ (err 0.125)
G ₃		$\alpha = 2.56$ (err 0.071)

잘 맞힌 (오류가 작은) 모형의 표가 더 무겁다 . 최종 예측 = $\text{sign}(\sum \alpha_m G_m(x))$. (α 는 뒤의 장난감 예제에서 계산한 값)

두 번의 가중치 : 관측치 가중치 w_i (학습할 때 · 어려운 문제에 집중) + 모형 가중치 α_m (결합할 때 · 잘하는 모형에 큰 표)

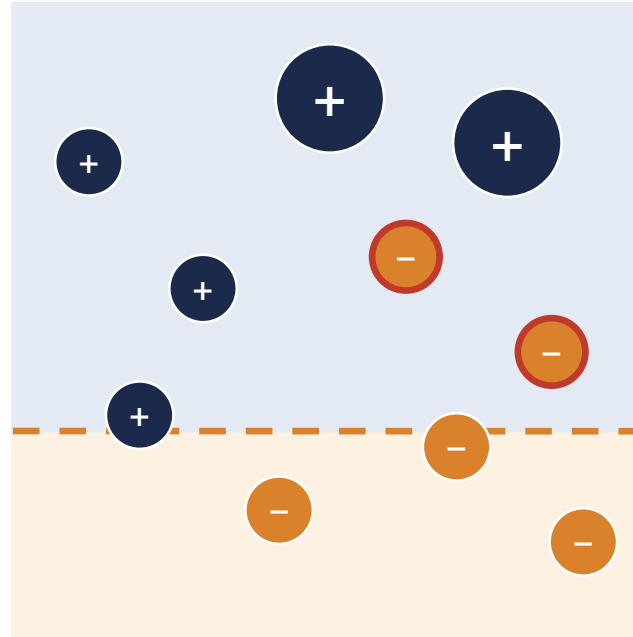
장난감 예제 : 10 개 관측치 , stump 3 개

Round 1



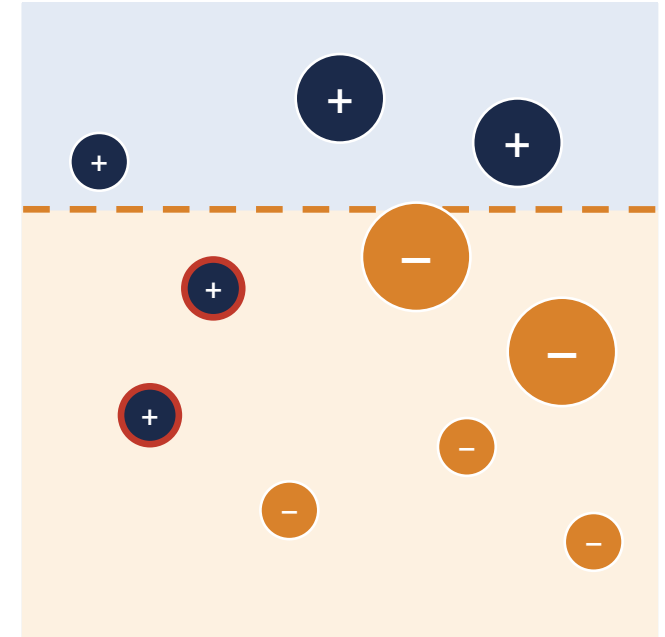
G₁: $X_1 < 0.36 \rightarrow +1$
 err = 0.2 $\alpha = 1.39$

Round 2



G₂: $X_2 < 0.325 \rightarrow -1$
 err = 0.125 $\alpha = 1.95$

Round 3

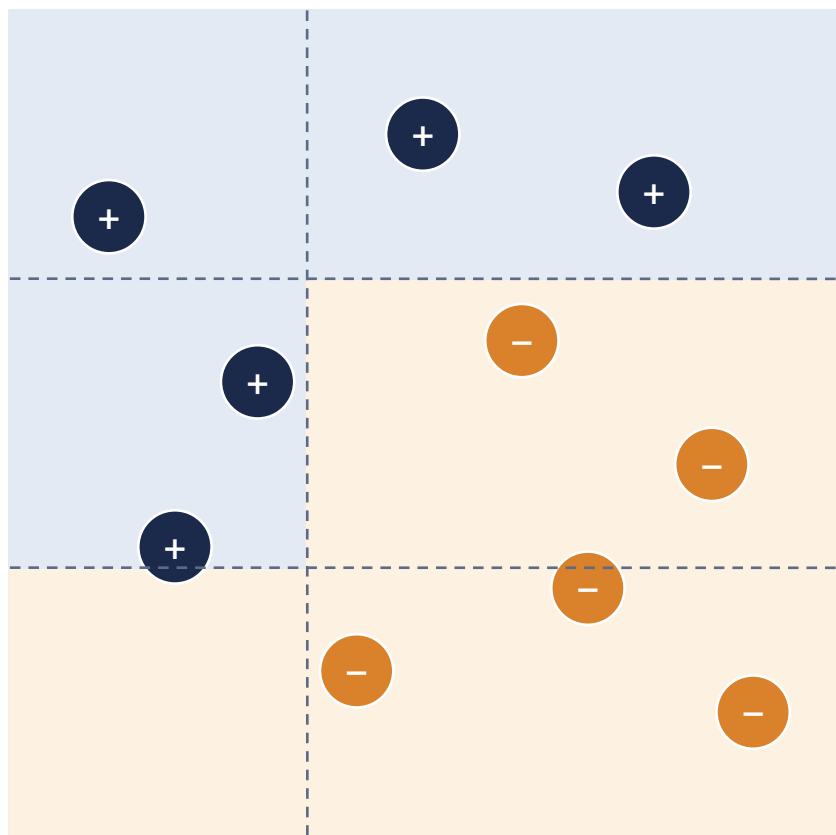


G₃: $X_2 < 0.675 \rightarrow -1$
 err = 0.071 $\alpha = 2.56$

● 원의 크기 = 그 라운드의 관측치 가중치 w_i ○ 빨간 테두리 = 이번 모형이 틀린 관측치 $\rightarrow$ 다음 라운드에서 커진다

세 개의 약한 분류기를 가중합하면

$$G(x) = \text{sign}[1.39 \cdot G_1 + 1.95 \cdot G_2 + 2.56 \cdot G_3]$$



파랑 = +1 영역, 주황 = -1 영역

개별 stump

각각 10 개 중 2 개를 틀림 (가중치 기준 오류 0.20, 0.13, 0.07)

가중합 결과

10 개 모두 정확히 분류 — 직선 하나로는 불가능한 경계를 계단 모양으로 만들어 냄

핵심

단순한 모형도 “틀린 곳에 집중 + 가중 결합” 으로 복잡한 경계를 표현

AdaBoost 의 성질과 주의점

1 학습 오차가 빠르게 감소

약한 학습기가 무작위보다 조금만 나아도 학습 데이터 오차는 지수적으로 줄어든다

2 주로 bias 를 줄인다

단순한 (높은 bias) stump 들을 쌓아 복잡한 모형을 만든다 — Bagging/RF 와 반대 방향의 처방

3 잡음 · 이상치에 민감

라벨이 잘못된 관측치는 계속 틀리므로 가중치가 끝없이 커진다 → 그 점에 끌려다님

4 M 을 너무 크게 하면

결국 과대적합 가능 → 반복 수 M 은 CV 로 선택

그런데 AdaBoost 는 왜 잘 될까 ? → 2000 년 Friedman·Hastie·Tibshirani 가 “최적화” 관점으로 다시 해석

AdaBoost 를 다시 보면 : 손실을 조금씩 줄여 가는 “덧셈 모형”

$$f_m(x) = f_{m-1}(x) + \alpha_m G_m(x)$$

지금까지 만든 모형에 “새 모형 하나” 를 더해 가는 과정 (forward stagewise additive modeling)

AdaBoost = 지수 손실을 줄이는 과정

$L(y, f) = \exp(-y \cdot f(x))$. 매 단계 이 손실이 가장 많이 줄도록 G_m 과 α_m 을 고른 것과 정확히 같다

관측치 가중치의 정체

$w_i \propto \exp(-y_i f(x_i))$: 현재 모형이 틀린 ($y_i f < 0$) 관측치일수록 크다 = “ 이 관측치에서 손실이 크고, 줄일 여지가 크다”

일반화 : 손실함수를 우리가 원하는 것 (제곱오차 , 로지스틱 ...) 으로 바꾸면 ? → 손실을 줄이는 “방향” 을 찾는 문제 = 최적화

다시 local search: 어디선가 시작해서, 손실이 작은 쪽으로

지난 시간 : Stepwise (이산적인 공간)

모형 = 변수 부분집합 . 이웃 = 변수 하나 추가 / 제거 . AIC 가 가장 작아지는 이웃으로 한 걸음씩 이동 , 개선이 없으면 정지

오늘 : 연속적인 공간이라면 ?

움직일 대상이 숫자 (파라미터 , 예측값) 라서 이웃이 무한히 많다 → 어느 방향으로 , 얼마나 움직일지를 정해야 한다

1 시작점

어딘가에서 시작 (예 : 모든 예측 = $\bar{y}$)

2 방향

손실이 가장 빨리 줄어드는 쪽
= 음의 기울기 ($-\text{gradient}$)

3 보폭

한 번에 얼마나 갈까 = 학습률
(learning rate) ν

4 정지

손실이 더 이상 줄지 않거나 ,
정해진 횟수 도달 (CV 로 결정)

이산이든 연속이든 같은 local search: “ 지금 위치에서 더 좋아지는 쪽으로 한 걸음씩 ” . 이제 연속 공간에서의 기본 방법을 보자

기본 알고리즘 : 최급강하법 (steepest descent)

$$\theta_{\text{new}} = \theta_{\text{old}} - \nu \cdot \nabla L(\theta_{\text{old}})$$

방향 : $-\nabla L$

기울기 (gradient) 는 손실이 가장 빨리 커지는 방향 $\rightarrow$ 그 반대로 가면 가장 빨리 줄어든다

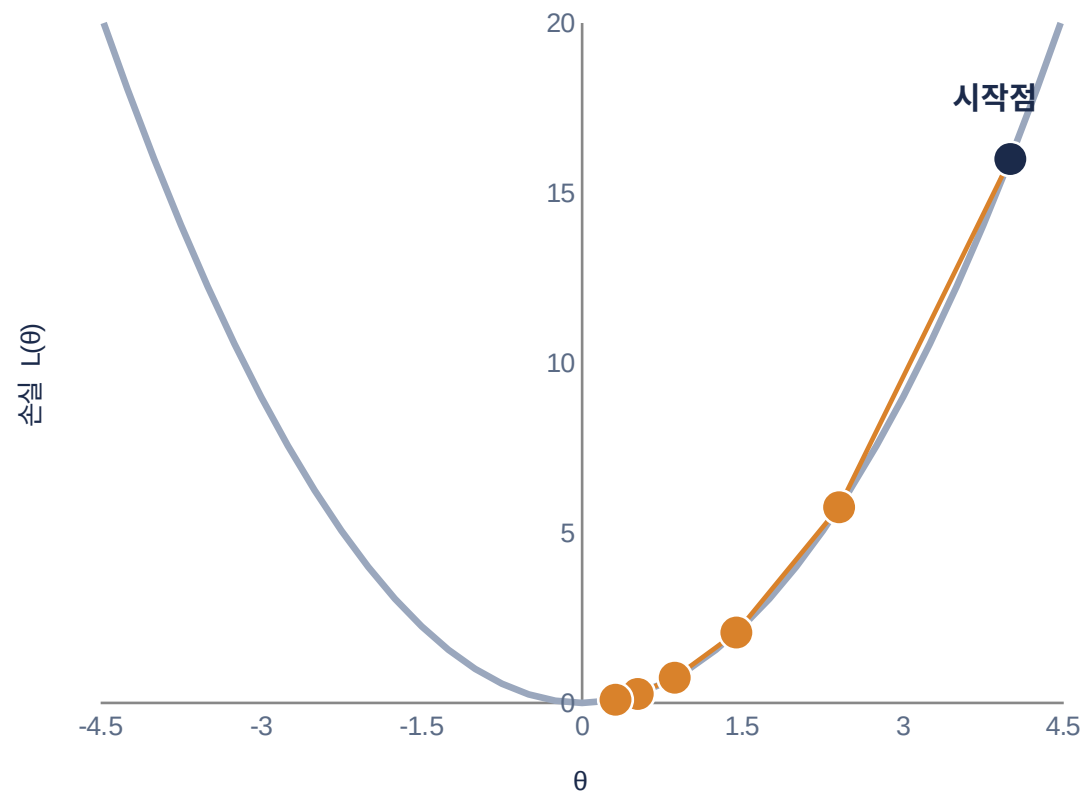
보폭 : ν (learning rate)

기울기에 곱하는 작은 수 . 한 걸음의 크기를 조절

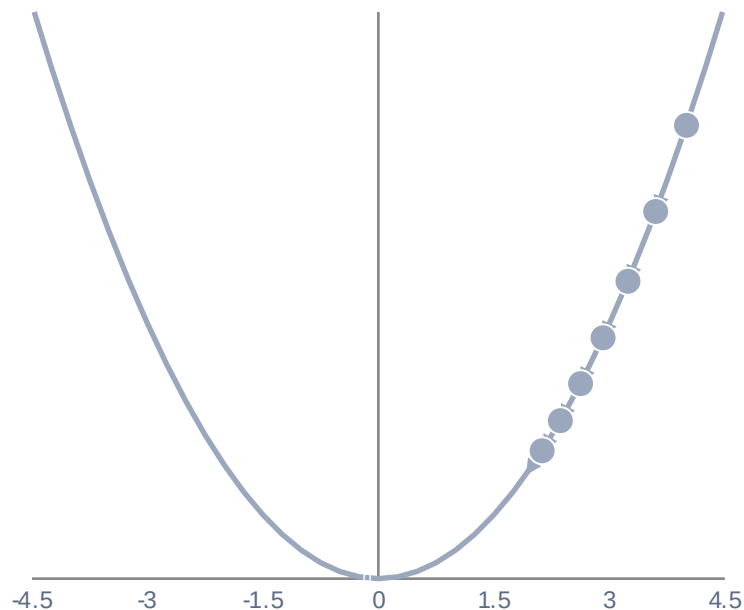
다른 방법도 있다

Newton 법 , momentum, Adam 등 다양한 최적화 알고리즘 . 하지만 기본은 steepest descent

예: $L(\theta) = \theta^2$, $\theta_0 = 4$, $\nu = 0.2 \rightarrow \theta \leftarrow \theta - 0.2 \cdot 2\theta = 0.6\theta : 4 \rightarrow 2.4 \rightarrow 1.44 \rightarrow 0.86 \rightarrow \dots$

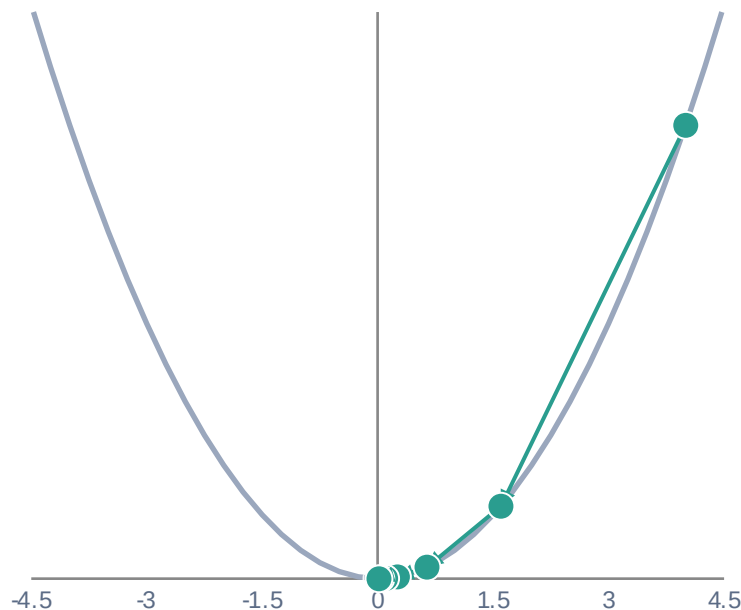


보폭 (learning rate) 의 선택



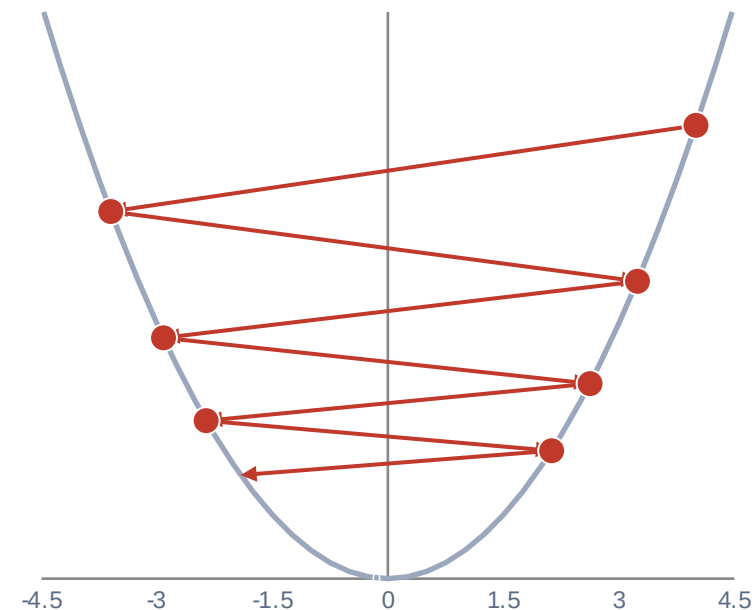
너무 작다 ($v = 0.05$)

안전하지만 아주 느리다



적당 ($v = 0.3$)

빠르게 최솟값 근처로



너무 크다 ($v = 0.95$)

최솟값을 지나쳐 좌우로 튈다

Boosting에서는 보통 작은 v ($0.01 \sim 0.1$) + 많은 반복 M . 작은 보폭이 과대적합도 막아 준다 (뒤의 시뮬레이션)

Boosting 에서 “움직이는 것” 은 예측값 자체

파라미터 θ 대신 , 학습 데이터 n 개 점에서의 예측값을 움직인다고 생각하자

관측치 i	y_i	현재 예측 $f(x_i)$	$-\partial L / \partial f(x_i)$
1	3.1	2.4	0.7
2	0.8	1.5	-0.7
3	2.2	2.1	0.1
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	1.9	1.2	0.7

* 제곱오차 손실일 때의 예시 수치

1. 방향 계산

각 관측치에서 손실의 음의 기울기 $r_i = -\partial L(y_i, f(x_i)) / \partial f(x_i)$ (pseudo-residual)

2. 문제

이 방향은 학습 데이터 n 개 점에서만 알 수 있다 . 새로운 x 에서는 ? 그대로 따라가면 학습 데이터를 외울 뿐

3. 해결 : 트리로 근사

x 로 r_i 를 예측하는 작은 트리 $h_m(x)$ 를 적합 $\rightarrow$ 방향을 “일반화”

4. 이동

$$f_m(x) = f_{\{m-1\}}(x) + v \cdot h_m(x)$$

Gradient Boosting = 함수 공간에서의 경사하강법 . 매 단계의 “한 걸음” 이 바로 트리 하나 (Friedman, 2001)

Gradient Boosting 알고리즘 (Friedman, 2001)

- 1 시작점

$$f_0(x) = \operatorname{argmin}_c \sum L(y_i, c)$$

회귀 (제곱오차) 면 $f_0 = \bar{y}$
- 2 방향

$$r_{im} = -[\partial L(y_i, f(x_i)) / \partial f(x_i)] \text{ at } f = f_{\{m-1\}}$$

pseudo-residual 계산
- 3 트리 적합

$$r_{im} \text{ 을 반응변수로 작은 회귀나무 } h_m(x) \text{ 적합}$$

깊이 1~6 정도의 약한 트리
- 4 이동

$$f_m(x) = f_{\{m-1\}}(x) + \nu \cdot h_m(x)$$

ν : learning rate (shrinkage)
- 5 반복

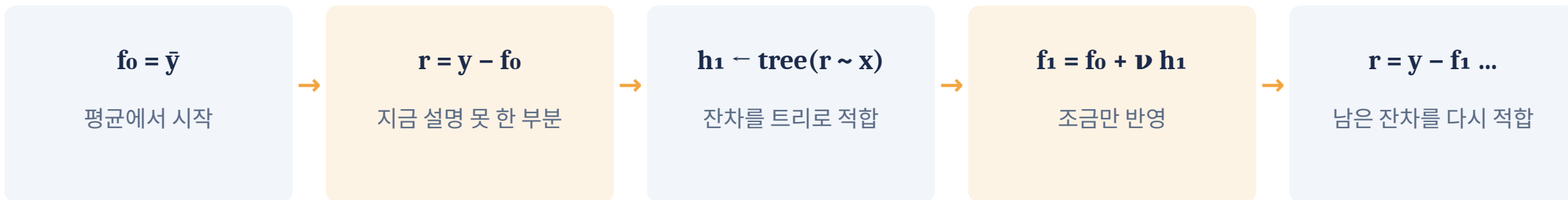
$$m = 1, \dots, M \text{ 동안 } 2\sim4 \text{ 반복}$$

M 은 CV / 검증오차로 결정

손실함수 L 만 바꾸면 회귀 · 분류 · 순위 등 어떤 문제에도 같은 틀을 쓸 수 있다

회귀 : 손실을 미분하면 “잔차”가 된다

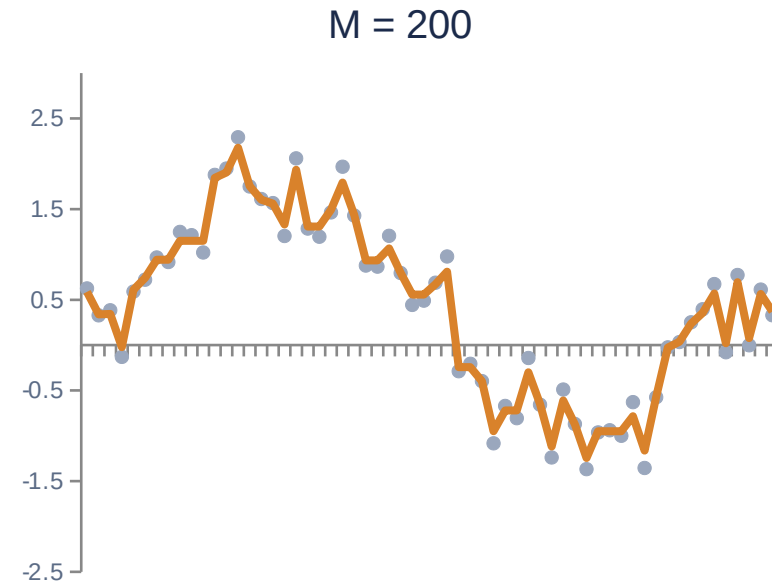
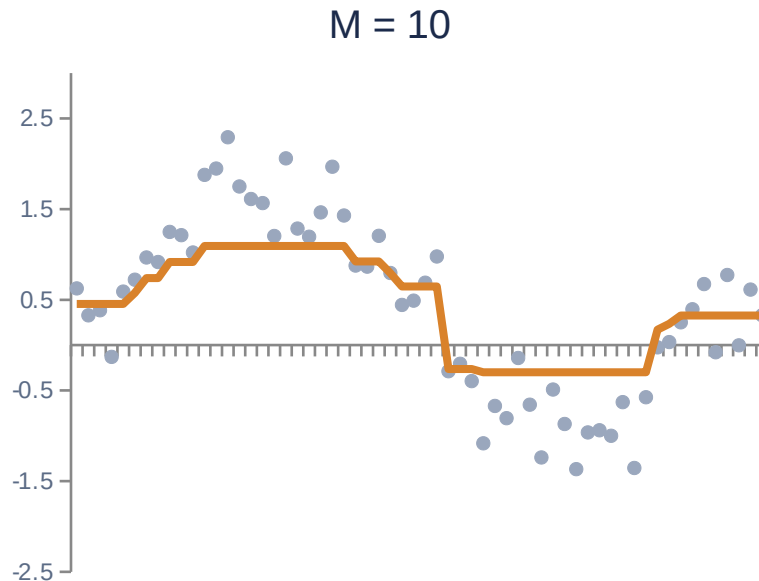
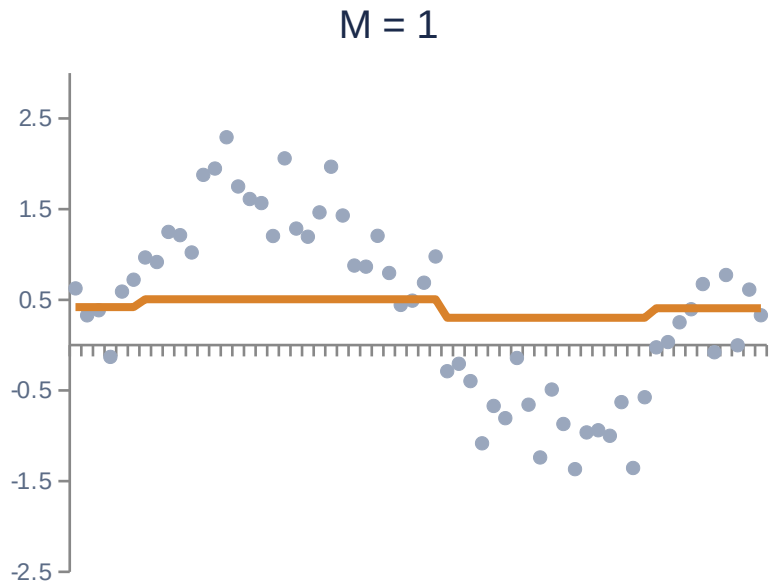
$$L = \frac{1}{2} (y - f)^2 \Rightarrow -\partial L / \partial f = y - f = \text{잔차 (residual)}$$



“잔차를 계속 다시 적합 (refitting) 한다”

직관 : 첫 트리가 큰 흐름을 잡고, 다음 트리들은 “아직 못 맞힌 부분” 만 조금씩 메워 간다. 경사하강법의 한 걸음 = 잔차 방향으로 ν 만큼 이동

잔차를 반복 적합하면 모형이 데이터를 따라간다



● 데이터 — Boosting 예측

M = 1

평균 근처에서 거의 움직이지 않음 ($v = 0.1$ 이라 한 걸음이 작다)

M = 10

큰 흐름 (사인 모양) 을 따라가기 시작

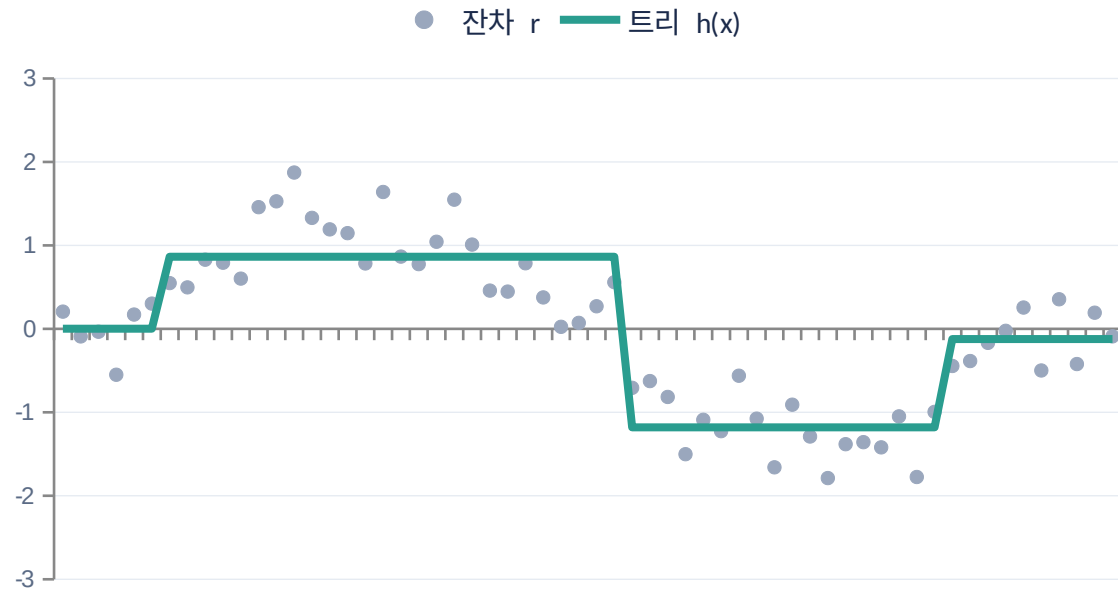
M = 200

세부 패턴까지 포착 . 더 키우면 잡음까지 따라갈 위험 → M 선택 중요

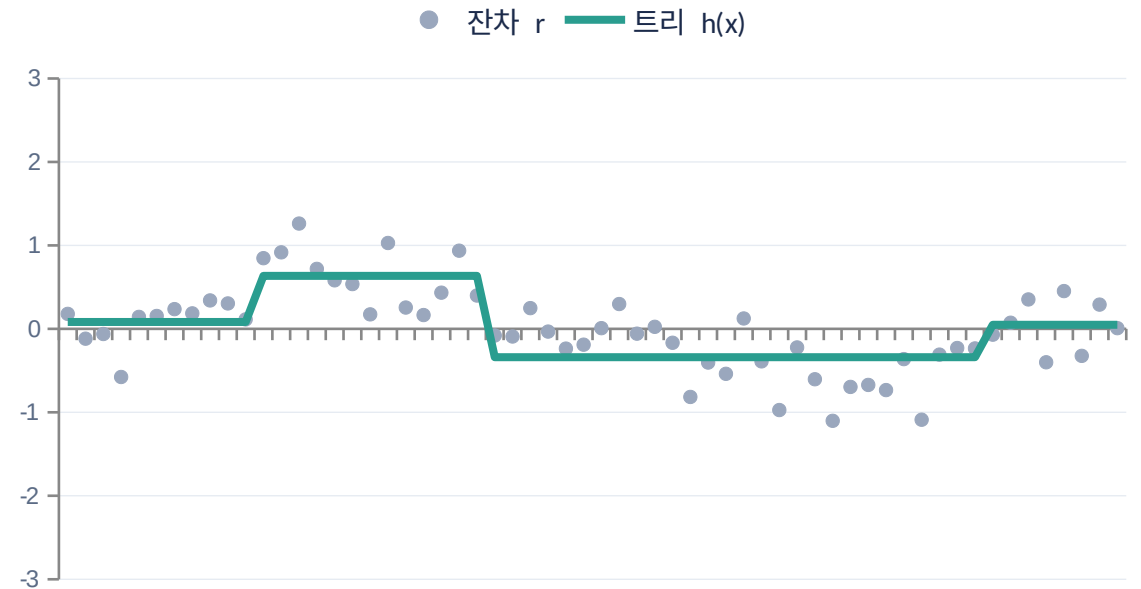
$y = 1.5 \cdot \sin(2\pi x) + x + \text{잡음}$, $n = 60$. 깊이 2 트리, $v = 0.1$ (Python 으로 직접 계산)

한 걸음 들여다보기 : 잔차에 트리를 적합한다

Step 1: 처음 잔차 $r = y - \bar{y}$ 와 적합된 트리 h_1



Step 10: 남은 잔차는 작아지고 , 트리는 남은 패턴을 적합



트리 $h(x)$ = 잔차 (= 음의 기울기) 를 x 의 함수로 근사한 “방향” . 모형은 이 방향으로 $v = 0.1$ 만큼만 이동한다

분류 : 로지스틱 손실을 미분하면 ?

$Y \in \{0, 1\}$, $f(x) =$ 로그 오즈 (log-odds), $p(x) = P(Y = 1 | x) = 1 / (1 + e^{(-f(x))})$

$$L(y, f) = -[y \cdot \log p + (1 - y) \cdot \log(1 - p)] \quad (\text{이항 deviance, 음의 로그우도})$$

$$-\partial L / \partial f = \mathbf{y - p(x)} = \text{관측값} - \text{예측 확률}$$

y = 1 인 관측치 : **r = 1 - p(x)**

$P(Y=1|x)$ 가 1 이 되도록 끌어올린다 → “1 - $P(Y=1|X)$ 를 줄이려 한다”

y = 0 인 관측치 : **r = 0 - p(x) = -p(x)**

$P(Y=1|x)$ 가 0 이 되도록 끌어내린다

분류에서도 결국 “잔차” 를 적합한다

y	현재 $\hat{p}(x)$	잔차 $r = y - \hat{p}$	의미
1	0.95	0.05	거의 맞힘 → 조금만 수정
1	0.20	0.80	크게 틀림 → 강하게 끌어올림
0	0.70	-0.70	크게 틀림 → 강하게 끌어내림
0	0.10	-0.10	거의 맞힘 → 조금만 수정

회귀와 같은 구조

회귀 : $y - \hat{f}$ (값의 잔차), 분류 : $y - \hat{p}$ (확률 척도의 잔차). 둘 다 “관측값 - 현재 예측”

AdaBoost 와 연결

$|y - \hat{p}|$ 가 큰 관측치 = 잘 못 맞힌 관측치 → 다음 트리가 더 집중 . AdaBoost 의 “틀린 관측치 가중치 ↑” 와 같은 정신

정리 : Gradient Boosting 은 어떤 손실을 쓰든 “현재 모형이 가장 크게 틀리고 있는 곳” 을 다음 트리가 보완하도록 방향을 잡는다 . 트리는 로그 오즈 $f(x)$ 에 더해지고 , 확률은 $p = 1/(1+e^{(-f)})$ 로 얻는다

손실함수에 따라 달라지는 “잔차”

문제	손실함수 $L(y, f)$	음의 기울기 $-\partial L/\partial f$	R gbm distribution
회귀	제곱오차 $\frac{1}{2}(y - f)^2$	$y - f$ (잔차)	"gaussian"
회귀 (이상치에 강함)	절대오차 $ y - f $	$\text{sign}(y - f)$	"laplace"
분류 (0/1)	이항 deviance	$y - p(x)$	"bernoulli"
분류 ($-1/+1$)	지수 손실 $\exp(-y \cdot f)$	$y \cdot \exp(-y \cdot f) \rightarrow \text{AdaBoost}$	"adaboost"
개수 자료	포아송 deviance	$y - e^f$	"poisson"

AdaBoost 도 이 틀의 특수한 경우 : 지수 손실의 기울기 크기 $\exp(-y \cdot f)$ 가 바로 AdaBoost 의 관측치 가중치 . 알고리즘은 하나 , 손실함수만 바꾸면 된다

Gradient Boosting 의 튜닝 모수

1 M (n.trees)

반복 수 = 트리 개수 . RF 와 달리 너무 크면 과대적합 → CV 로 선택

2 ν (shrinkage)

학습률 . 작을수록 성능이 좋아지는 경향이지만 더 많은 M 필요 (보통 0.01~0.1)

3 트리 깊이 (interaction.depth)

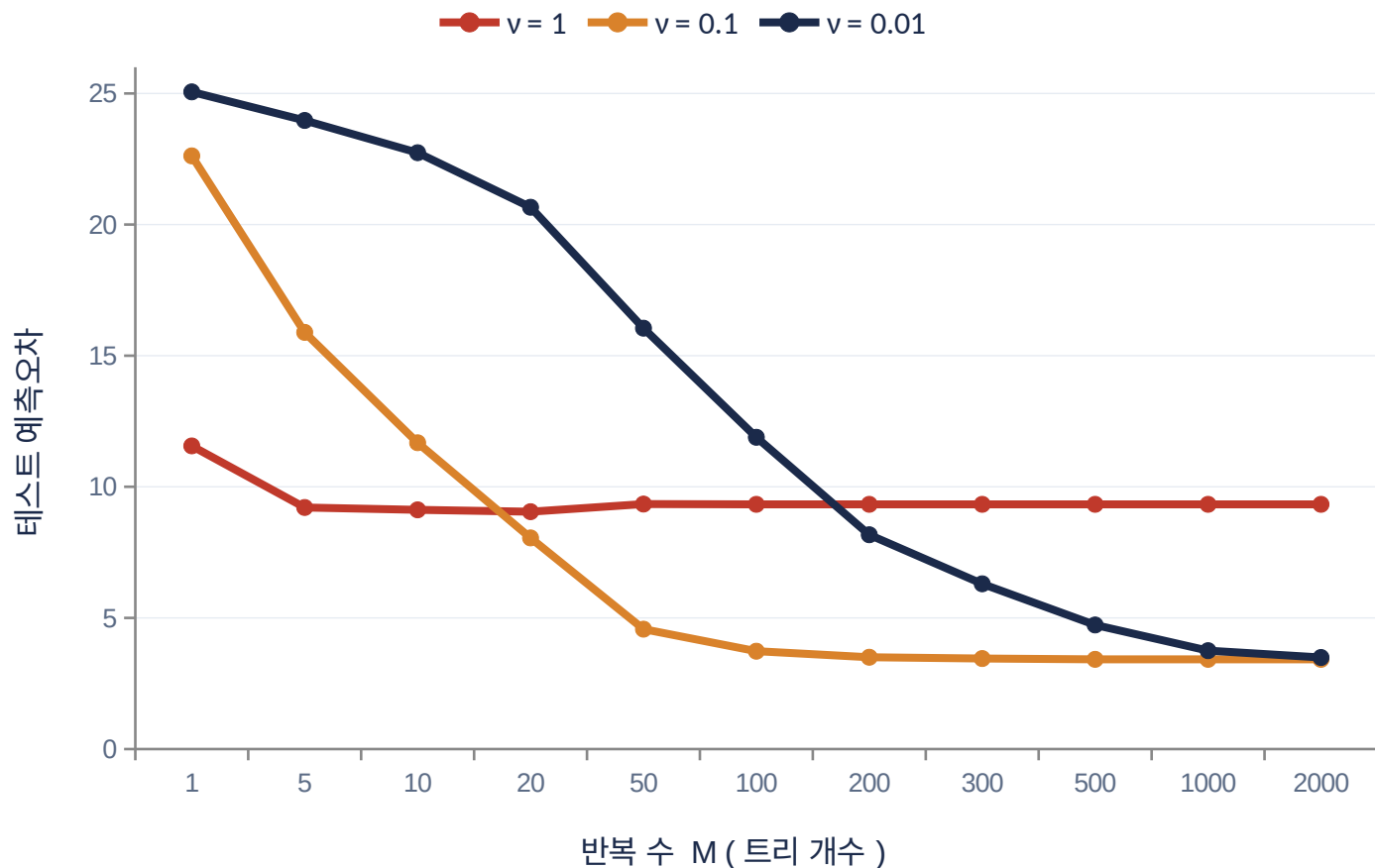
1 = stump (상호작용 없는 덧셈 모형), 2 = 2 차 상호작용 ...
보통 1~6

4 부표본 비율 (bag.fraction)

매 단계 학습 데이터의 일부 (예 : 50%) 만 무작위 사용 →
Stochastic Gradient Boosting, 분산 감소

ν 와 M 은 짝 : 보폭을 줄이면 더 많이 걸어야 한다 . 먼저 ν 를 작게 정하고 , M 은 CV(gbm.perf) 로 정한다

학습률 v 와 반복 수 M 에 따른 테스트 오차



$v = 1$

$M \approx 20$ 에서 최저 (≈ 9.0) 후 오히려 증가 ·
학습오차는 0 → 과대적합

$v = 0.1$

천천히 내려가 ≈ 3.41 까지 · 큰 보폭보다 훨씬 좋다

$v = 0.01$

$M = 2000$ 에서도 아직 내려가는 중 → 더 많은 M 필요

Friedman #1 데이터 (앞 강의 RF 시뮬레이션과 같은 설정, $n = 300$), 깊이 3 트리. 같은 데이터에서 RF 의 오차는 약 5.8 → 잘 튜닝된 boosting 이 더 정확. x 축 간격은 균등하지 않음.

R 실습 : adabag(AdaBoost.M1) 와 gbm

R CONSOLE

```
# AdaBoost.M1 ( 분류 , 예 : iris)
> library(adabag)
> ada <- boosting(Species ~ ., data=iris, mfinal=100,
+               coeflearn="Freund") #  $\alpha = \log((1-\text{err})/\text{err})$ 
> ada$weights          # 모형별 가중치  $\alpha_m$ 

# Gradient Boosting ( 회귀 , 조류 데이터 a1)
> library(gbm)
> set.seed(1234)
> gb <- gbm(a1 ~ ., data=clean.algae[,1:12], distribution="gaussian",
+   n.trees=3000, shrinkage=0.01, interaction.depth=3,
+   bag.fraction=0.5, cv.folds=5)
> best <- gbm.perf(gb, method="cv") # CV 로 최적 M
> summary(gb, n.trees=best)          # 변수 중요도 (VIP)
> plot(gb, i.var="P04", n.trees=best) # PDP
> predict(gb, newdata=test, n.trees=best)
```

coeflearn

"Breiman"(기본): $\alpha = \frac{1}{2} \cdot \log((1-e)/e)$,
 "Freund": 강의 식과 동일

distribution

"gaussian" · "bernoulli" · "adaboost" ...
 손실함수 선택

gbm.perf()

CV 오차가 최소인 반복 수 M

분류 (0/1)

distribution="bernoulli", predict(...,
 type="response") 로 확률

Bagging · Random Forest vs Boosting

	Bagging / Random Forest	Boosting
모형 생성	병렬 · 서로 독립	순차 · 앞 모형에 의존
개별 트리	깊은 트리 (낮은 bias, 높은 분산)	작은 트리 (높은 bias, 낮은 분산)
주로 줄이는 것	Variance	Bias (그리고 일부 variance)
결합	동일 가중 평균 / 투표	가중합 (α_m , 또는 $v \cdot h_m$)
트리 수 증가	과대적합 없음	과대적합 가능 $\rightarrow$ CV
튜닝 부담	적음 (mtry)	큼 (M, v, 깊이, 부표본)
잡음 · 이상치	비교적 강함	민감 (손실함수 선택으로 완화)

해석은 두 방법 모두 VIP(변수 중요도) 와 PDP(부분 의존도) 로

Boosting 의 장점 , 주의점 , 그리고 현대적 구현

장점

- 표 형태 데이터에서 최고 수준의 예측 정확도
- 손실함수를 바꿔 다양한 문제에 적용
- VIP·PDP 로 해석 가능

주의점

- M 이 크면 과대적합 $\rightarrow$ CV/ 조기 종료
- 튜닝할 모수가 많다
- 순차적이라 병렬화가 RF 보다 어렵다
- 잡음이 많은 라벨에 민감

현대적 구현

- XGBoost: 2 차 미분 (뉴턴 단계) + 정규화
- LightGBM: 대용량 데이터용 고속 구현
- CatBoost: 범주형 변수 처리 강화
- 모두 기본 뼈대는 오늘 배운 gradient boosting

한 줄 요약 : Boosting = 손실함수의 음의 기울기 (잔차) 를 작은 트리로 근사해 , 조금씩 (v) 더해 가는 함수 공간의 경사하강법

SUMMARY

오늘의 핵심 정리

1 순차적 학습

앞 모형이 틀린 부분을 다음 모형이 보완 . 약한 모형을 쌓아 bias 를 줄인다

2 AdaBoost.M1

틀린 관측치 가중치 $\uparrow$ (학습), 잘한 모형일수록 큰 표 α_m (결합)

3 Local search

시작점에서 손실이 줄어드는 방향 ($-\text{gradient}$) 으로 보폭 ν 만큼 이동 : steepest descent

4 Gradient Boosting

음의 기울기 = 잔차 (회귀 : $y - f$, 분류 : $y - p$). 잔차를 트리로 반복 적합